

Messagerie Instantanee

Application Client-Serveur TCP/IP

Guide de Presentation

Python - TCP/IP - Tkinter - SQLite - Threading

Projet de Fin de Module - Architecture Reseau & Programmation

1. Introduction

Ce projet est une application de messagerie instantanee complete developpee en Python. Elle repose sur une architecture client-serveur avec communication TCP/IP, une interface graphique Tkinter, une base de donnees SQLite, et un chiffrement des mots de passe avec bcrypt.

L'objectif etait de creer un systeme de messagerie professionnel permettant :

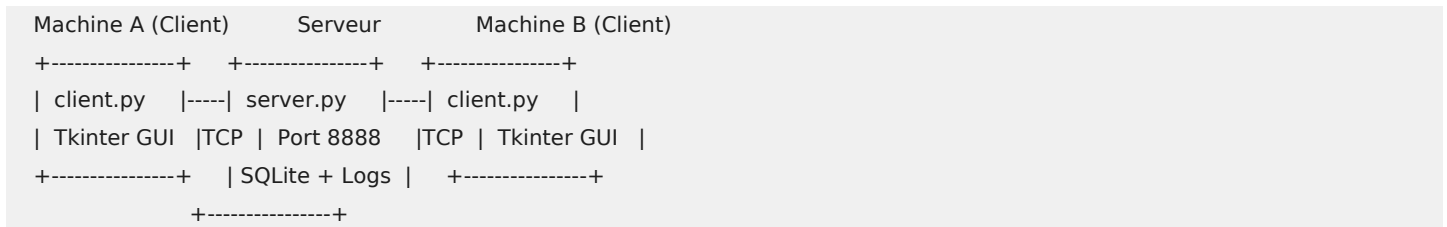
- La messagerie privee et de groupe en temps reel
- Le transfert de fichiers entre utilisateurs
- La gestion de profils utilisateur completes
- Un systeme d'administration et de securite
- Une journalisation exhaustive de toutes les actions

2. Architecture Technique

2.1 Modele Client-Serveur TCP/IP

L'application suit un modele client-serveur classique : un serveur centralise les connexions, achemine les messages et gere la base de donnees. Chaque client maintient une connexion TCP persistante avec le serveur.

Schema de l'architecture :



2.2 Pourquoi TCP (pas UDP) ?

Le choix de TCP (SOCK_STREAM) plutot que UDP (SOCK_DGRAM) est un choix d'architecture fondamental, justifie par les exigences d'une messagerie :

Critere	TCP (choisi)	UDP (rejete)
Livraison	Garantie (acquittements)	Non garantie (perte possible)
Ordre	Preserve	Non preserve
Connexion	Persistante (suivi d'etat)	Sans connexion
Bidirectionnel	Simultane (full-duplex)	Unidirectionnel

Dans une messagerie, la fiabilite prime sur la vitesse : on ne peut pas se permettre de perdre un message ou de le recevoir dans le desordre. TCP repond parfaitement a ce besoin.

2.3 Architecture Multithread

Le serveur utilise un modele multithread pour gerer les connexions simultanees :

- 1 thread principal qui accepte les nouvelles connexions
- 1 thread dedie par client connecte pour gerer sa communication
- Les threads sont en mode 'daemon' pour un arret propre du serveur

Code d'acceptation des connexions :

```

while self.running:
    client, addr = self.server_socket.accept()
    logging.info(f"Nouvelle connexion de {addr}")
    t = threading.Thread(target=self.handle_client,
                        args=(client, addr))
    t.daemon = True
    t.start()
  
```

2.4 Protocole Textuel

Le protocole de communication est textuel : chaque commande est une chaine de caracteres terminee par un saut de ligne (\n). Le serveur utilise un buffer pour reassembler les paquets TCP qui peuvent etre fragmentes.

```

buffer = ""
while True:
    chunk = client.recv(1024).decode()
    buffer += chunk
    while '\n' in buffer:
        line, buffer = buffer.split('\n', 1)
        self.process_line(line)
  
```

3. Fonctionnalités Complètes

3.1 Authentification et Sécurité

- **Inscription** : Inscription avec login et mot de passe
- **Connexion** : Connexion avec vérification bcrypt (hash + sel aléatoire)
- Compatibilité ascendante : migration automatique anciens mots de passe vers bcrypt
- Protection : tentatives échouées loggées en WARNING

3.2 Messagerie Privée

- Envoi et réception de messages en temps réel
- Affichage en bulles (vert = moi, blanc = autre)
- Horodatage automatique de chaque message
- Indicateur de lecture : ✓ (lu) / (non lu)
- Suppression de ses propres messages (individuel ou tout)
- Raccourci Entrée pour envoyer, Shift+Entrée pour nouvelle ligne
- Scroll automatique vers le bas

3.3 Messagerie de Groupe

- Création de groupes avec nom personnalisé
- Ajout de membres avec notification automatique
- Envoi de messages à tous les membres connectés
- Affichage des noms des expéditeurs dans le groupe
- Suppression de messages de groupe
- Retrait d'un membre (createur seulement)
- Quitter un groupe / Supprimer un groupe (createur)

3.4 Transfert de Fichiers

- Envoi de fichiers en discussion privée et en groupe
- Fichiers convertis en hexadécimal pour transport TCP
- Stockage côté serveur dans le dossier received_files/
- Cache local chez chaque client (received_files/)
- Téléchargement automatique via GET_FILE si fichier absent du cache
- Ouverture avec l'application par défaut du système (xdg-open)
- Correction : socket.sendall() garantit l'envoi complet des gros fichiers

3.5 Profils Utilisateur

- Bio, photo de profil, téléphone, email, adresse
- Éducation et travail
- Consultation du profil des autres utilisateurs
- Modification en temps réel de chaque champ
- Photo de profil avec prévisualisation circulaire (Pillow)

3.6 Indicateur de Frappe (Typing)

- Affichage en temps réel : 'X tape...'
- Arrêt automatique après 2 secondes d'inactivité
- Version groupe : 'Quelqu'un écrit...'

3.7 Administration

- Interface de gestion des utilisateurs (admin seulement)
- Suppression definitive d'un compte utilisateur
- Verification du statut admin avant chaque action sensible
- Tentatives non autorisees loggees et bloquees

3.8 Interface Utilisateur

- Fenetre divisee en 3 panneaux redimensionnables
- Design sobre inspire de WhatsApp (bulles vertes/blanches)
- Rafraichissement automatique toutes les 2 secondes
- Indicateur de connexion en temps reel (vert/rouge)
- Gestion propre de la fermeture (envoi de QUIT)

4. Base de Donnees SQLite

La base de donnees contient 5 tables relationales :

Table : users

Stocke les informations des utilisateurs : id, login, password (hash bcrypt), bio, photo, phone, email, address, education, work, is_admin, created_at.

Table : messages

Messages prives : id, sender_id, receiver_id, content, sent_at, is_read. Les fichiers sont stockes comme des messages avec le format :nom:taille.

Tables : groups, group_members, group_messages

groups : id, name, creator_id, created_at

group_members : group_id, user_id (cle composite)

group_messages : id, group_id, sender_id, content, sent_at

Les fichiers ne sont PAS stockes dans la BDD pour des raisons de performance. Seule une reference textuelle (:nom_fichier:taille) est stockee dans le champ 'content'. Les fichiers reels sont sur le disque du serveur dans received_files/.

Schema Relationnel

```
users ----< messages      (sender_id, receiver_id)
users ----< groups        (creator_id)
users >----< group_members (user_id)
groups >----< group_members (group_id)
groups ----< group_messages (group_id)
users ----< group_messages (sender_id)
```

5. Securite

5.1 Chiffrement des Mots de Passe (bcrypt)

Les mots de passe sont haches avec bcrypt, un algorithme de hachage adaptatif qui utilise un sel aleatoire pour chaque mot de passe. Meme si la base de donnees est compromise, les mots de passe restent proteges.

```
# Inscription - hachage du mot de passe
hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt()).decode()
cursor.execute("INSERT INTO users (login, password) VALUES (?, ?)",
               (login, hashed))

# Connexion - verification
bcrypt.checkpw(password.encode(), stored_pw.encode())
```

5.2 Administration

L'administrateur est identifie par le champ is_admin = 1 dans la table users. Seul l'admin peut supprimer des comptes utilisateur. Toute tentative non autorisee est loggee et bloquee avec un message d'erreur.

5.3 Journalisation (Logging)

Toutes les actions sont journalisees avec timestamp et niveau de severite :

Evenement	Niveau	Exemple
Connexion reussie	INFO	alice authentifie depuis 192.168.1.42
Tentative echouee	WARNING	Tentative echouee depuis 192.168.1.42
Envoi message	INFO	192.168.1.42 - SEND bob Bonjour
Suppression non auto.	WARNING	bob a tente de supprimer le msg 6 sans autorisation
Erreur reseau	ERROR	Erreur avec 192.168.1.42: Connection reset

6. Logs et Preuves de Fonctionnement

Le systeme de logs est un element cle du projet. Il prouve que l'application a ete testee intensivement et qu'elle fonctionne correctement.

6.1 Statistiques

Le fichier de logs (logs/serveur.log) contient plus de 176 000 lignes, temoignant de sessions de test multiplies avec plusieurs utilisateurs simultanés.

6.2 Exemples de Logs

```
2026-07-11 13:00:01 - INFO - Serveur démarre sur 0.0.0.0:8888
2026-07-11 13:00:01 - INFO - Nouvelle connexion de (127.0.0.1, 59786)
2026-07-11 13:00:24 - WARNING - Tentative echouee depuis (127.0.0.1, 59786)
2026-07-11 13:00:35 - INFO - alice authentifie depuis (127.0.0.1, 59786)
2026-07-11 13:02:08 - INFO - (127.0.0.1, 59786) - SEND bob Bonjour Bob !
2026-07-11 13:40:33 - WARNING - bob a tente de supprimer le message 6
2026-07-11 16:42:16 - INFO - alice a supprime tous ses messages (5 msg)
```

Chaque log contient : un timestamp precis, un niveau de severite (INFO, WARNING, ERROR), et le detail de l'action. Cela permet une tracabilite complete et facilite le debogage.

7. Probleme Rencontre et Correction

Le bug : transfert de fichiers entre machines distantes

Lors des tests avec deux machines differentes via Tailscale, les fichiers etaient bien envoyes et affiches dans la discussion, mais le destinataire ne pouvait pas les ouvrir en cliquant dessus. Sur la meme machine, tout fonctionnait.

Cause racine

Le probleme venait de l'utilisation de `socket.send()` pour envoyer les donnees hexadecimales des fichiers. Pour un fichier de 5 Mo, les donnees hexa representent environ 10 Mo de texte. Or, `send()` n'envoie pas forcément toutes les donnees en un seul appel : il retourne le nombre d'octets reellement envoyes, et pour les gros volumes, ce nombre peut etre inferieur a la taille totale.

```
# AVANT (bug) : socket.send() n'envoie pas tout !
sock.send(f"...{hex_data}...\n".encode()) # perte de donnees !

# Le destinataire recevait des donnees tronquees
# bytes.fromhex() echouait silencieusement (except: pass)
# Le fichier n'etait jamais sauvegarde localement
```

Solution : `socket.sendall()`

Le remplacement de `send()` par `sendall()` sur tous les appels lies aux fichiers resout le probleme. `sendall()` boucle en interne jusqu'a ce que toutes les donnees soient transmises.

```
# APRES (corrige) : sendall() garantit l'envoi complet
sock.sendall(f"...{hex_data}...\n".encode()) # donnees integrales !
```

Corrections appliquees

Fichier	Ligne	Changement
server.py	560	<code>send()</code> -> <code>sendall()</code> envoi fichier prive
server.py	578	<code>send()</code> -> <code>sendall()</code> reponse GET_FILE
server.py	690	<code>send()</code> -> <code>sendall()</code> envoi fichier groupe
client.py	1499	<code>send()</code> -> <code>sendall()</code> envoi GROUP_FILE
client.py	1504	<code>send()</code> -> <code>sendall()</code> envoi FILE
client.py	1529	<code>send()</code> -> <code>sendall()</code> requete GET_FILE

Ameliorations complementaires

- Ajout d'un indicateur 'Telechargement en cours...' dans l'interface
- Messages d'erreur explicites (au lieu de `except: pass` silencieux)
- Gestion des erreurs serveur (Fichier introuvable, Erreur de lecture)

8. Demonstration en Direct

Etapes pour la demo :

1. Demarrer le serveur : `python3 server.py`
2. Ouvrir 2 clients : `python3 client.py` (dans 2 terminaux)
3. Se connecter avec `alice/1234` et `bob/5678`
4. Envoyer un message prive de alice a bob
5. Creer un groupe, ajouter bob, envoyer un message groupe
6. Envoyer un fichier (PDF) de alice a bob
7. Chez bob, cliquer sur le fichier -> s'ouvre automatiquement
8. Montrer les logs en direct : `tail -f logs/serveur.log`
9. Consulter le profil : clic sur un nom -> voir les informations
10. Taper un message pour montrer l'indicateur 'tape...'

9. Questions / Reponses Anticipees

Q : Pourquoi ne pas avoir utilise UDP pour etre plus rapide ?

Dans une messagerie, la fiabilite prime sur la vitesse. On ne peut pas se permettre de perdre un message. TCP garantit la livraison et l'ordre, ce qui est essentiel pour une conversation coherente.

Q : Comment gerez-vous la concurrence ?

Chaque client est gere dans un thread separe. La base de donnees SQLite gere les acces concurrents avec son propre verrouillage au niveau fichier.

Q : Qu'est-ce qui vous a pris le plus de temps ?

Le transfert de fichiers entre machines distantes. On a decouvert que `socket.send()` n'envoie pas tout d'un coup pour les gros volumes. La correction avec `sendall()` a resolu le probleme. On a aussi ajoute un indicateur de telechargement et des messages d'erreur explicites.

Q : Comment prouver que l'application a ete testee ?

On a plus de 176 000 lignes de logs avec timestamps, adresses IP, et niveaux de severite — connexions, messages, tentatives echouees, tout est trace et horodate.

Q : Securite des mots de passe ?

Bcrypt avec sel aleatoire pour chaque mot de passe. Meme si la base de donnees fuit, les mots de passe restent proteges. Les anciens mots de passe stockes en clair sont automatiquement migres vers bcrypt lors de la premiere connexion.

Q : Quelles sont les limites et perspectives d'amelioration ?

Limites actuelles : pas de chiffrement TLS (les donnees circulent en clair sur le reseau), pas de rate limiting (blocage apres tentatives echouees), pas de persistance des fichiers apres envoi (pas de cleanup).

Ameliorations possibles : ajout de TLS/SSL, chiffrement de bout en bout, interface web, notification mobile, stockage cloud des fichiers, microservices.